

C'est qui ce ZIG ?

Didier Plaindoux



@dplaindoux@functional.cafe







Langage de programmation système, moderne et robuste
qui propose de la méta-programmation via l'exécution de
code à la compilation !



Langage de programmation système, moderne et robuste
qui propose de la méta-programmation via l'exécution de
code à la compilation !

Langage sans flux de contrôle caché



Langage de programmation système, moderne et robuste
qui propose de la méta-programmation via l'exécution de
code à la compilation !

Langage sans flux de contrôle caché

Langage sans allocations mémoire cachées



Langage de programmation système, moderne et robuste
qui propose de la méta-programmation via l'exécution de
code à la compilation !

Langage sans flux de contrôle caché

Langage sans allocations mémoire cachées

Langage sans préprocesseur, ni macros

Survol du langage ZIG



Types natifs

Types natifs

Nombres (entier et flottant)

i8, u8, i16, u16, i32, u32, i64, u64, isize, usize

f16, f32, f64, f80, f128

Types natifs

Nombres (entier et flottant)

i8, u8, i16, u16, i32, u32, i64, u64, isize, usize

f16, f32, f64, f80, f128

Compatibilité ABI avec C

c_char, c_short, c_ushort, c_int, c_uint, c_long etc.

Types natifs

Nombres (entier et flottant)

i8, u8, i16, u16, i32, u32, i64, u64, isize, usize

f16, f32, f64, f80, f128

Compatibilité ABI avec C

c_char, c_short, c_ushort, c_int, c_uint, c_long etc.

Autres ...

bool, void, anyopaque, noreturn, type, anytype, anyerror

Fonction

```
fn nom(p1: t1, ..., p1: tn) r {  
    ...  
}
```

Les paramètres sont passés par valeur

Fonction

```
fn nom(p1: t1, ..., p1: tn) r {  
    ...  
}
```

Les paramètres sont passés par valeur

les primitives sont copiées et

Fonction

```
fn nom(p1: t1, ..., p1: tn) r {  
    ...  
}
```

Les paramètres sont passés par valeur

les primitives sont copiées et

pour les données composites cela dépend

Optionnel ?T

Soit la valeur **null** soit une valeur de type **T**

```
const a : ?u32 = ...;
```

```
const value1 = a orelse ...;
```

```
const value2 = a orelse unreachable; // panique si [null]
```

```
const value3 = a.?; // équivalent à [a orelse unreachable]
```

Optionel ?T

Soit la valeur **null** soit une valeur de type **T: ?T**

```
const a : ?u32 = ...;
```

```
if (a != null) {  
    ... // L'appel à [a.?] est sécurisé  
} else {  
    ...  
}
```


Optionel ?T

Soit la valeur **null** soit une valeur de type **T**: **?T**

```
const a : ?u32 = ...;
```

```
if (a) |value| {  
    ...  
} else {  
    ...  
}
```

Optionnel ?T

Soit la valeur **null** soit une valeur de type **T**: **?T**

```
var a : ?u32 = ...;
```

```
if (a) |*value| {  
    ...  
} else {  
    ...  
}
```

Enumération

Ensemble restreint de valeurs nommées

```
const Couleur = enum { rouge, noir };  
const Enseigne = enum { pique, trefle, carreau, coeur };
```

Enumération

Traitement par filtrage exhaustif

```
const Couleur = enum { rouge, noir };  
const Enseigne = enum { pique, trefle, carreau, coeur };  
  
fn couleur(e: Enseigne) Couleur {  
    return switch (e) {  
        .pique, .trefle => .noir,  
        .carreau, .coeur => .rouge,  
    };  
}
```

Enumération

Colocaliser représentation et comportements

```
const Couleur = enum { rouge, noir };  
const Enseigne = enum {  
    pique, trefle, carreau, coeur,  
  
    fn couleur(e: Enseigne) Couleur {  
        ...  
    }  
};  
// Enseigne.couleur(.pique) ou Enseigne.pique.couleur()
```

Enumération

Typage et référence au type courant

```
const Couleur = enum { rouge, noir };  
const Enseigne = enum {  
    pique, trefle, carreau, coeur,  
  
    fn couleur(e: @This()) Couleur {  
        ...  
    }  
};  
// Enseigne.couleur(.pique) ou Enseigne.pique.couleur()
```

Erreur

Enumérations spécifiques dédiées aux erreurs

```
const A = error{ NotDir, PathNotFound };  
const B = error{ OutOfMemory, PathNotFound };  
const C = A || B;  
// C = error{ OutOfMemory, NotDir, PathNotFound }
```

Pas de notion d'exceptions !

Erreur

Traitement des erreurs

```
const r = operation catch comportement_en_cas_d_erreur;
```

```
const r = operation catch |e| comportement_en_cas_d_erreur;
```

```
const r = operation catch |e| return e;
```

```
const r = try operation; // Sucre syntaxique
```


Fonction

Retourner des erreurs

```
fn nom(p1: t1, ..., p1: tn) !r { // inférence
    ...
}
```

```
fn nom(p1: t1, ..., p1: tn) A!r {
    ...
}
```

```
fn nom(p1: t1, ..., p1: tn) error{OutOfMemory}!r {
    ...
}
```

Fonction

Retourner des erreurs

```
const MathError = error{DivideByZero};

fn divide(a: f64, b: f64) MathError!f64 {
    if (b == 0) {
        return MathError.DivideByZero;
    }

    return a / b;
}
```

Erreur et diagnostique

Type énuméré donc sans valeurs

Approches possibles:

- Remplacer **E!R** par un type algébrique comme **Result**
- Avoir un paramètre en écriture pour le diagnostique

Tableaux

Tableau statique

```
const hello: [5]u8 = [_]u8{ 'h', 'e', 'l', 'l', 'o' };
```

Sous Tableau ou “Slice”

```
const he: []const u8 = hello[0..2];
```

Tableau dynamique avec `std.ArrayList`

Vecteur (support SIMD)

```
const v1: @Vector(4, f32) = .{ 1.0, 2.0, 3.0, 4.0 };
```

Contrôle

Contrôle

```
if (condition) true_branch [else false_branch]
```

Contrôle

```
if (condition) true_branch [else false_branch]
```

```
for(items) |item| { ... }
```

Contrôle

```
if (condition) true_branch [else false_branch]
```

```
for(items) |item| { ... }
```

```
for(items) |item, index| { ... }
```


Contrôle

```
if (condition) true_branch [else false_branch]
```

```
for(items) |item| { ... }
```

```
for(items) |item, index| { ... }
```

```
for(items1,items2) |item1, item2| { ... }
```

Contrôle

```
if (condition) true_branch [else false_branch]
```

```
for(items) |item| { ... }
```

```
for(items) |item, index| { ... }
```

```
for(items1,items2) |item1, item2| { ... }
```

```
while(i < 10) { ... }
```

Contrôle

```
if (condition) true_branch [else false_branch]
```

```
for(items) |item| { ... }
```

```
for(items) |item, index| { ... }
```

```
for(items1,items2) |item1, item2| { ... }
```

```
while(i < 10) { ... }
```

```
while (i < 10) : (i += 1) { ... }
```

Contrôle

```
if (condition) true_branch [else false_branch]
```

```
for(items) |item| { ... }
```

```
for(items) |item, index| { ... }
```

```
for(items1,items2) |item1, item2| { ... }
```

```
while(i < 10) { ... }
```

```
while (i < 10) : (i += 1) { ... }
```

Tout est expression

```
_ = if (condition) 1 else 2;
```

```
_ = for (5..10) |i| i else @"return": {  
    break :@"return" 10;  
};
```

```
var i = 0;  
_ = while (i < 10) : (i += 1) {} else @"return": {  
    break :@"return" 10;  
};
```

Structure

Type produit hérité du langage C

Structure

Type produit hérité du langage C

```
const Point = struct {  
    x: i32,  
    y: i32,  
};
```

Structure

Type produit hérité du langage C

```
const Point = struct {  
    x: i32,  
    y: i32,  
};
```

```
fn init(x: i32, y: i32) Point {  
    return Point { .x = x, .y = y };  
}
```


Structure

Type produit hérité du langage C

```
const Point = struct {  
    x: i32,  
    y: i32,  
};
```

```
fn init(x: i32, y: i32) Point {  
    return Point { .x = x, .y = y };  
}
```

Structure

Type produit hérité du langage C

```
const Point = struct {  
    x: i32,  
    y: i32,  
};
```

```
fn init(x: i32, y: i32) Point {  
    return .{ .x = x, .y = y };  
}
```

Structure

Colocaliser représentation et comportements

```
const Point = struct {  
    x: i32,  
    y: i32,  
  
    fn init(x: i32, y: i32) Point {  
        return .{ .x = x, .y = y };  
    }  
};
```

Structure

Typage et référence au type courant

```
const Point = struct {  
    x: i32,  
    y: i32,  
  
    fn init(x: i32, y: i32) @This() {  
        return .{ .x = x, .y = y };  
    }  
};
```

Structure

Structure

Tout fichier source est une Structure

Structure

Tout fichier source est une Structure

x: i32,

y: i32,

Structure

Tout fichier source est une Structure

```
x: i32,  
y: i32,
```

```
fn init(x: i32, y: i32) @This() {  
    return .{ .x = x, .y = y };  
}
```


Structure

Tout fichier source est une Structure

x: i32,

y: i32,

```
fn init(x: i32, y: i32) @This() {  
    return .{ .x = x, .y = y };  
}
```

Structure

Tout fichier source est une Structure

x: i32,

y: i32,

```
fn init(x: i32, y: i32) @This() {  
    return @This(){ .x = x, .y = y};  
}
```

Union

Type somme hérité du langage C

```
const Number = union {  
    integer: i64,  
    float: f64,  
};
```

Soit un integer soit un float

Union

Accès sécurisé à la compilation

```
const Number = union { integer: i64, float: f64, };
```

```
test "Qui ne compile pas :)" {  
    const n = NumberSimple{ .integer = 42 };  
    try std.testing.expectEqual(42, n.float);  
}
```

error: access of union field 'float' while field 'integer' is active

Union

Accès sécurisé à l'exécution

```
const Number = union { integer: i64, float: f64, };
```

```
fn integer(n: Number) i64 {  
    return n.integer;  
}
```

panic: access of union field 'integer' while field 'float' is active

Union Étiquetée

Union Étiquetée

Inspirée des langages fonctionnels

Union Étiquetée

Inspirée des langages fonctionnels

- Etiquetage Implicite**

Union Étiquetée

Inspirée des langages fonctionnels

- Etiquetage Implicite**
- Etiquetage Explicite**

Union Étiquetée

Inspirée des langages fonctionnels

- **Etiquetage Implicite**
- **Etiquetage Explicite**
- **Filtrage de Motif**

Union

Étiquetage implicite

```
const Number = union(enum) { integer: i64, float: f64, };
```

```
fn zeroInteger() Number {  
    return .{ .integer = 0 };  
}
```

```
fn zeroFloat() Number {  
    return .{ .float = 0 };  
}
```

Union

Étiquetage explicite

```
const Label = enum { entier, flottant };
```

```
const Number = union(Label) { entier: i64, flottant: f64, };
```

```
fn zero(label: Label) Number {  
    return if (label == .entier) .{ .entier = 0 }  
        else .{ .flottant = 0 };  
}
```

Union

Cas du Filtrage de Motif

```
const Number = union(enum) { entier: i64, flottant: f64, };

fn increment(number: Number) Number {
    return switch (number) {
        .entier => |n| .{ .entier = n + 1 },
        .flottant => |f| .{ .flottant = f + 1 },
    };
}
```

Union

Colocaliser représentation et comportements

```
const Number = union(enum) {  
    entier: i64,  
    flottant: f64,  
  
    fn increment(number: Number) Number {  
        return ...  
    }  
};
```

Union

Typage et référence au type courant

```
const Number = union(enum) {  
    entier: i64,  
    flottant: f64,  
  
    fn increment(self: @This()) @This() {  
        return ...  
    }  
};
```

Union

Cas des types récurifs

```
const Naturel = union(enum) {  
    Zero,  
    Succ: Naturel,  
};
```

```
error: type 'Naturel' depends on itself for field declared here  
    Succ: Naturel,  
          ^~~~~~
```

Il faut passer par les pointeurs.

Pointeur

Définition de type : `*T` ne peut jamais être nul

Syntaxe : `&` pour l'adresse et `.*` pour déréférencer

Implicite : `.*` optionel lors d'accès (fonction ou champ)

Arithmétique indirecte : par coercion de type

```
var x: i32 = 1234;
```

```
const ptr = &x;
```

```
ptr.* = 5678; // Modification de la valeur
```

```
ptr += 1; // Erreur de compilation
```

Gestion de la mémoire dans ZIG



Pile ou Tas ?

La Pile (Stack)

Taille connue et durée de vie liée au bloc de base

Portée : Destruction dès que l'on sort du bloc de base

Performance : Allocation efficace mais taille limitée

Contrainte : La taille doit être connue à la compilation

Pile ou Tas ?

Le Tas (Heap)

Taille dynamique ou durée de vie indépendante de la pile

Philosophie Zig : Pas d'allocateur global caché

Passage d'allocateur : Allocator comme paramètre

Contrainte : Libération explicite (cf. fuite mémoire)

Allocation sur la Pile

Allocation sur la Pile

```
const Point = struct { x: i32, y: i32 };
```

Allocation sur la Pile

```
const Point = struct { x: i32, y: i32 };
```

```
pub fn main() void {  
    const p1 = Point { .x = 0, .y = 0};  
}
```

Allocation sur la Pile

```
const Point = struct { x: i32, y: i32 };

pub fn main() void {
    const p1 = Point { .x = 0, .y = 0 };
    {
        const p2 = Point { .x = 42, .y = 42 };
        ...
    } // -> p2 fin de portée donc libéré
```


Allocation sur la Pile

```
const Point = struct { x: i32, y: i32 };

pub fn main() void {
    const p1 = Point { .x = 0, .y = 0 };
    {
        const p2 = Point { .x = 42, .y = 42 };
        ...
    } // -> p2 fin de portée donc libéré
    ...
}
```

Allocation sur le Tas

Oui mais lequel ?

Allocation sur le Tas

Oui mais lequel ?

- DebugAllocator**

Allocation sur le Tas

Oui mais lequel ?

- **DebugAllocator**
- **SafeAllocator**

Allocation sur le Tas

Oui mais lequel ?

- **DebugAllocator**
- **SafeAllocator**
- **ArenaAllocator**

Allocation sur le Tas

Oui mais lequel ?

- **DebugAllocator**
- **SafeAllocator**
- **ArenaAllocator**
- **PageAllocator**

Allocation sur le Tas

Oui mais lequel ?

- **DebugAllocator**
- **SafeAllocator**
- **ArenaAllocator**
- **PageAllocator**

...

Allocation

Allocation

```
pub fn main() !void {
```

Allocation

```
pub fn main() !void {  
    var heap = std.heap.DebugAllocator(.{}){};  
    defer _ = heap.deinit();  
    const allocator = heap.allocator();  
    const array = try allocator.alloc(u8, 10);  
    defer allocator.free(array);  
}
```

Allocation

```
pub fn main() !void {  
    var heap = std.heap.DebugAllocator(.{}){};  
    defer _ = heap.deinit();  
    const allocator = heap.allocator();  
    const array = try allocator.alloc(u8, 10);  
    defer allocator.free(array);  
    ...  
}
```

Allocation

```
pub fn main() !void {  
    var heap = std.heap.DebugAllocator(.{}){};  
    defer _ = heap.deinit();  
    const allocator = heap.allocator();  
    const array = try allocator.alloc(u8, 10);  
    defer allocator.free(array);  
    ...  
}
```

Fuite mémoire

```
pub fn main() !void {  
    var heap = std.heap.DebugAllocator(.{}){};  
    defer _ = heap.deinit();  
    const allocator = heap.allocator();  
    const array = try allocator.alloc(u8, 10);  
    ...  
}
```

error(DebugAllocator): memory address 0x10caa0000 leaked:
 const array = try allocator.alloc(u8, 10);
 ^

“Use-After-Free”

“Use-After-Free”

```
test "Use after free" {  
    var heap = std.heap.DebugAllocator(.{}){};  
    defer _ = heap.deinit();  
    const allocator = heap.allocator();
```

“Use-After-Free”

```
test "Use after free" {  
    var heap = std.heap.DebugAllocator(.{}){};  
    defer _ = heap.deinit();  
    const allocator = heap.allocator();  
  
    const score_ptr = try allocator.create(u32);
```


“Use-After-Free”

```
test "Use after free" {  
    var heap = std.heap.DebugAllocator(.{}){};  
    defer _ = heap.deinit();  
    const allocator = heap.allocator();  
  
    const score_ptr = try allocator.create(u32);  
    allocator.destroy(score_ptr);  
}
```

“Use-After-Free”

```
test "Use after free" {  
    var heap = std.heap.DebugAllocator(.{}){};  
    defer _ = heap.deinit();  
    const allocator = heap.allocator();  
  
    const score_ptr = try allocator.create(u32);  
    allocator.destroy(score_ptr);  
  
    score_ptr.* = 666; // Use-After-Free here
```

“Use-After-Free”

```
test "Use after free" {  
    var heap = std.heap.DebugAllocator(.{}){};  
    defer _ = heap.deinit();  
    const allocator = heap.allocator();  
  
    const score_ptr = try allocator.create(u32);  
    allocator.destroy(score_ptr);  
  
    score_ptr.* = 666; // Use-After-Free here  
}
```

“Use-After-Free”

“Use-After-Free”

Execution avec le DebugAllocator :

Segmentation fault at address 0x109a20000

use-after-free.zig:13:5: 0x1010c10c1 in main

```
    score_ptr.* = 666;
```

```
    ^
```

“Use-After-Free”

Execution avec le DebugAllocator :

```
Segmentation fault at address 0x109a20000  
use-after-free.zig:13:5: 0x1010c10c1 in main  
    score_ptr.* = 666;  
    ^
```

Solutions idiomatiques :

- utilisation du defer ou
- typer avec un optionel et mettre le pointeur à null.

“Stack Buffer Overflow”

Écriture en dehors de la structure prévue

```
fn foo(input: []const u8) void {  
    var buffer: [8]u8 = undefined;  
    @memcpy(buffer[0..], input);  
}  
  
pub fn main() void {  
    foo("Plus de 8 caractères!"); // 22 caractères  
}
```

“Stack Buffer Overflow”

Execution interrompue :

```
thread 1332888 panic: ... arguments have non-equal lengths
stack-buffer-overflow.zig:5:19: 0x10230955f in foo
@memcpy(buffer[0..], input);
                ^
```

```
stack-buffer-overflow.zig:9:8: 0x102309314 in main
foo("Plus de 8 caractères!");
```

Détection des débordements (modes de compilation)

Voir aussi Stack canaries, ASLR et DEP/NX.

“Stack Buffer Overflow”

```
fn foo(input: *const [8]u8) void {  
    var buffer: [8]u8 = undefined;  
    @memcpy(buffer[0..], input);  
}
```

```
pub fn main() void {  
    foo("Plus de 8 caractères!"); // 22 caractères  
}
```

expected type ‘const [8]u8’, found ‘const [22:0]u8’

“Double Free”

“Double Free”

```
var heap = std.heap.DebugAllocator(.{}){};  
defer _ = heap.deinit();  
const allocator = heap.allocator();
```

“Double Free”

```
var heap = std.heap.DebugAllocator(.{}){};  
defer _ = heap.deinit();  
const allocator = heap.allocator();
```

```
var number = try allocator.create(i32);  
allocator.destroy(number);
```

“Double Free”

```
var heap = std.heap.DebugAllocator(.{}){};  
defer _ = heap.deinit();  
const allocator = heap.allocator();
```

```
var number = try allocator.create(i32);  
allocator.destroy(number);
```

```
allocator.destroy(number);
```

“Double Free”

```
var heap = std.heap.DebugAllocator(.{}){};  
defer _ = heap.deinit();  
const allocator = heap.allocator();  
  
var number = try allocator.create(i32);  
allocator.destroy(number);  
  
allocator.destroy(number);
```

“Double Free”

```
var heap = std.heap.DebugAllocator(.{}){};  
defer _ = heap.deinit();  
const allocator = heap.allocator();
```

```
var number = try allocator.create(i32);  
allocator.destroy(number);
```

```
allocator.destroy(number);
```

Solution idiomatique :

- utilisation du `defer` ou `typer` avec un optionel.

Méta-programmation en ZIG



Comptime vs. Runtime

Code exécuté à la compilation

Systeme de type strict et duck typing avec anytype

Code exécuté au runtime

Systeme de type strict uniquement

Type comme valeur

Citoyen de 1ère classe à la compilation

Évaluation à la compilation via `comptime`

Fonction génératrice de type

Type comme valeur

```
const Nat : type = union(enum) {  
    Zero, Succ: *const Nat,  
  
    fn toInt(self: @This()) u32 {  
        return switch(self) {  
            .Zero => 0,  
            .Succ => |p| 1 + p.toInt(), // ≡ 1 + p.*.toInt()  
        };  
    }  
}
```

Quantification universel $\forall x.T$

Pas de quantification universel au runtime

Évaluation à la compilation via `comptime`

Fonction génératrice de type

Type générique

Type générique évalué à la compilation

```
fn Maybe(comptime T: type) type {  
    return union(enum) {  
        None,  
        Some: T,  
  
        fn pure(t:T) @This() {  
            return .{ .Some = t };  
        }  
    };  
}
```

Génération de types

Pure : $\forall M : * \rightarrow * . \forall A : * . A \rightarrow M(A)$

```
fn Pure(comptime M: fn (type) type, comptime A: type) type {  
    return fn (A) M(A);  
}
```

Optional : $\forall A : * . A \rightarrow ? A$

```
fn Optional(comptime A: type) type { return ?A; }
```

Déclaration de types

$\text{Add_u32} = \text{u32} \rightarrow \text{u32} \rightarrow \text{u32}$

```
const Add_u32 = fn(u32) fn(u32) u32;
```

$\text{Pure} = \forall M : * \rightarrow * . \forall A : * . A \rightarrow M(A)$

```
const Pure = fn(M:fn(type) type) fn(A:type) fn(A) M(A);
```

error: use of undeclared identifier 'A'

```
const Pure: type = fn(M:fn(type) type) fn(A:type) fn(A) M(A);  
                                     ^
```

Déclaration de types dérivée

$\text{Pure} = \forall M : * \rightarrow * . \forall A : * . A \rightarrow M(A)$

```
fn Pure(M: fn(type) type) type {  
  const inner = struct {  
    fn run(A: type) fn(A) M(A) {  
      unreachable;  
    }  
  }.run;  
  
  return @TypeOf(inner);  
}
```


Quantification existentiel ?

$$\exists x.T(x) = \forall y.(\forall x.T(x) \rightarrow y) \rightarrow y$$

$$\exists x.T(x) = \forall y.(\forall x.T(x) \rightarrow y) \rightarrow y$$

$$\exists x.T(x) = \forall y.(\forall x.T(x) \rightarrow y) \rightarrow y$$

```
pub fn Exists(T:fn (type) type) type {
```

$$\exists x.T(x) = \forall y.(\forall x.T(x) \rightarrow y) \rightarrow y$$

```
pub fn Exists(T:fn (type) type) type {  
    return struct {  
        pub fn pack(X:type, value:T(X)) type {
```

$$\exists x.T(x) = \forall y.(\forall x.T(x) \rightarrow y) \rightarrow y$$

```
pub fn Exists(T:fn (type) type) type {  
    return struct {  
        pub fn pack(X:type, value:T(X)) type {  
            return struct {  
                // anytype at comptime capture  $\forall x.T(x) \rightarrow y$   
                pub fn unpack(Y:type, cont:anytype) Y {  
                    return cont(X, value);  
                }  
            };  
        }  
    };  
}
```

$$\exists x.T(x) = \forall y.(\forall x.T(x) \rightarrow y) \rightarrow y$$

Passage à la pratique

```
fn Opt(comptime X:type) type { return ?X; }  
fn run(comptime X:type, value: Opt(X)) void { ... }  
  
fn main() void {  
    const existential = Exists(Opt).pack(i32, null);  
    existential.unpack(void, run);  
}
```

Applicable uniquement à la compilation !

Conception logiciel et ZIG



Programmation Orientée Donnée

```
const Point = struct {  
    x: i32,  
    y: i32,  
  
    fn init(x: i32, y: i32) @This() {  
        return .{ .x = x, .y = y};  
    }  
    fn move(self: @This(), dx: i32, dy: i32) @This() {  
        return .{ .x = self.x + dx, .y = self.y + dy};  
    }  
};
```


Programmation Orientée Donnée

Function call or Static Dispatch?

```
fn main() void {  
    const zero = Point.init(0, 0);  
  
    // Classical functional call  
    const point1 = Point.move(zero, 42, 42);  
  
    // DOT syntax (syntactic sugar)  
    const point2 = zero.move(42, 42);  
}
```

Programmation Orientée Donnée

Types sommes et Filtrage de motif

```
const Number = union(enum) {  
    entier: i64, flottant: f64,  
  
    fn increment(self: @This()) @This() {  
        return switch (number) {  
            .entier => |n| .{ .entier = n + 1 },  
            .flottant => |f| .{ .flottant = f + 1 },  
        };  
    }  
};
```

Programmation par Abstraction

Encodage à l'aide de table virtuelle

```
const Figure: type = struct {  
    v_impl: *anyopaque,  
    v_surface: *const fn (self: *anyopaque) f64,  
  
    pub fn surface(self: @This()) f64 {  
        return self.v_surface(self.v_impl);  
    }  
  
    pub inline fn from(impl: anytype) @This() { ... }  
};
```

Programmation par Abstraction

Encodage à l'aide de table virtuelle

```
const Cercle = struct {  
    rayon: f64,  
  
    pub fn surface(self: @This()) f64 {  
        return std.math.pi * self.rayon * self.rayon;  
    }  
};
```

Instanciación : `Figure.from(&Cercle{ .rayon = 10 })`

Programmation par Modules

Definition de la signature de module **Stack**

```
fn Stack(comptime T:fn (type) type, comptime A:type) type {  
    return struct {  
        create: fn () callconv(."inline") T(A),  
        push: fn (Allocator, A, T(A)) anyerror!T(A),  
        peek: fn (T(A)) ?A,  
        pop: fn (Allocator, T(A)) Pair(?A, T(A)),  
    };  
}
```

Programmation par Modules

Mise en oeuvre avec les List

```
fn StackList(comptime A: type) type {  
    return struct {  
        inline fn create() List(A) {  
            return .nil();  
        }  
        ...  
    };  
}
```

Programmation par Modules

Un peu de Duck Typing à la compilation

```
fn implUsing(comptime T: type, comptime I: type) S {  
    var result: T = undefined;  
  
    const target_info = @typeInfo(T).@"struct"; // unsafe  
    inline for (target_info.fields) |f| {  
        @field(result, f.name) = @field(I, f.name);  
    }  
  
    return result;  
}
```

Programmation par Modules

Mise en pratique

Programmation par Modules

Mise en pratique

```
var heap = std.heap.DebugAllocator(.{}){};  
defer _ = heap.deinit();  
const allocator = heap.allocator();
```

Programmation par Modules

Mise en pratique

```
var heap = std.heap.DebugAllocator(.{}){};  
defer _ = heap.deinit();  
const allocator = heap.allocator();
```

```
const s = implUsing(Stack(List, u32), StackList(u32));  
const stack = try s.push(allocator, 1, S.create());  
defer s.deinit(allocator, stack);
```

Programmation par Modules

Mise en pratique

```
var heap = std.heap.DebugAllocator(.{}){};  
defer _ = heap.deinit();  
const allocator = heap.allocator();  
  
const s = implUsing(Stack(List, u32), StackList(u32));  
const stack = try s.push(allocator, 1, S.create());  
defer s.deinit(allocator, stack);  
  
try std.testing.expectEqual(1, s.peek(stack));
```

Programmation par Modules

Mise en pratique

```
var heap = std.heap.DebugAllocator(.{}){};  
defer _ = heap.deinit();  
const allocator = heap.allocator();  
  
const s = implUsing(Stack(List, u32), StackList(u32));  
const stack = try s.push(allocator, 1, S.create());  
defer s.deinit(allocator, stack);  
  
try std.testing.expectEqual(1, s.peek(stack));
```

Programmation par Modules

Est-ce pertinent ?

```
...  
const s = implUsing(Stack(List, u32), StackList(u32));  
// Expose le type interne ^^^^  
...
```

Abstraction via type existentiel limitée à comptime !

Utiliser les vtable et les types opaques en runtime

Programmation Fonctionnelle

Pas de fonction anonyme

Pas de fermeture (closure)

Ordre supérieur

- **Fonction qui manipule des fonctions**
- **Fonction qui retourne une fonction**

 **ZIG et C**



Utiliser des librairies C

Importation Intuitive

Utiliser des librairies C

Importation Intuitive

```
const ncurses = @cImport({  
    @cInclude("ncurses.h");  
});
```

Utiliser des bibliothèques C

Importation Intuitive

```
const ncurses = @cImport({  
    @cInclude("ncurses.h");  
});  
  
pub fn main() !void {  
    _ = ncurses.initscr();  
    defer _ = ncurses.endwin();  
    ...  
}
```

Mise à disposition à C

Exportation avec les types C dans Zig

```
extern fn min(a: c_int, b: c_int) callconv(.C) c_int {  
    return if (a < b) a else b;  
}
```

Declaration externe en C

```
extern int min(int a, int b);
```

Outillage  ZIG



Production de binaires

Une seule commande sans dépendance

Remplacement de C/C++

Traduction C vers Zig

Cross-Compilation native (inclus WASM)

Linker Réécrit de zéro (pas de ld ou lld)

Le Système de Build

Tout en Zig via un fichier `build.zig`

Gestionnaire de paquets via `build.zig.zon`

API riche

- **Générer du code**
- **Exécuter des tests**

Conclusion



Pourquoi choisir **ZIG** ?

Zig se positionne comme un “C robuste”

Interopérabilité “triviale” avec le C

Pas de flux de contrôle caché

Maîtrise totale de la mémoire et du matériel

Puissance de Comptime vs. Préprocesseur

Outillage moderne



C'est qui ce gonze ?

Systeme Distribué et modèle Acteur (Akawan Kaptngo)

Systeme Expert en Prolog (Crédifix)

Fonctions et Calcul des Ambiants (Ephel)

Libraries OCaml avec des amis (CargoCut)

C'est qui ce ZIG ?



Dépôt



Feedback

